



Information Retrieval, Word Embedding & Generative Models

IFN647 – Week 3

Professor Yuefeng Li
School of Computer Science
Queensland University of Technology

Learning Objectives

1

Topic ID	Key topics	Mapping to slides	Activities
1	Overview of IR Models	2-5	Q1
2	Vector Space Model	6-11	
3	Word embedding	12-15	Q6, Q7
4	Generative Models: Bayes classifier	16-19	Q2
5	Binary Independence Model	20-23	Q3
6	The BM25 Ranking Algorithm	24-27	
7	Abstract Model of Ranking for IR and ML	28-31	Q4, Q5
	References	32	

1. Overview of IR Models

- In Natural Language Processing (NLP), Information Retrieval (IR) systems are crucial for efficiently extracting relevant information from documents.
- IR models are crucial in search engines, recommendation systems, and chatbots.
- IR models provide a mathematical framework for defining the search process
 - includes explanation of assumptions and ranking.
- For a given query Q , an IR model finds relevant documents to answer Q .
- The evolution of information retrieval (IR) models has corresponded with sustained improvements in retrieval effectiveness, allowing systems to more reliably retrieve highly relevant document sets in response to query Q .
 - **Effectiveness** (quality): We want to be able to retrieve the most relevant set of documents possible for a query.
 - **Efficiency** (speed): We want to process queries from users as quickly as possible.
- The key research issue is theories about **relevance**.

Relevance

- Relevance is a complex concept that has been extensively studied.
 - It depends on multiple interacting factors.
 - Individuals often disagree when making relevance judgments.
- IR models make various assumptions about relevance to **simplify problem**
 - **Topical** vs. **user** relevance
 - A document is topically relevant to a query if it is judged to be on the same topic, i.e., the query and the document are about the same thing.
 - User relevance considers all the other factors that go into a user's judgment of relevance.
 - **Binary** vs. *multi-valued* relevance

Classical IR Models: Boolean Models

- **A Boolean Retrieval Model** - retrieves relevant documents using a set of Boolean logic operators, such as AND, OR, and NOT
 - Two possible outcomes for query processing
 - TRUE and FALSE
 - “exact-match” retrieval
 - simplest form of ranking
 - Query usually specified using Boolean operators
 - AND, OR, NOT,
 - Proximity operators (define constraints on the distance **between words**; e.g., requiring words to co-occur within a certain “window” (length) of text); and
 - Wildcard characters (define the minimum string match required for **a word**) are also commonly used in Boolean queries.
 - Example of queries for searching “President Lincoln”:
 - president AND lincoln AND NOT (automobile OR car)
 - president AND lincoln AND (biography OR life OR birthplace OR gettysburg) AND NOT (automobile OR car)



This will retrieve any document containing the words “president” and “lincoln”, along with any one of the words “biography”, “life”, “birthplace”, or “gettysburg” (and does not mention “automobile” or “car”).

Boolean Retrieval Summary

- Let $T = \{t_1, t_2, \dots, t_m\}$ be a set of *index terms*, where *each term* is a word or expression; and $D = \{D_1, D_2, \dots, D_n\}$ be a set of documents.
- $Q = (w_1 \vee w_2 \vee \dots) \wedge \dots \wedge (w_i \vee w_{i+1} \vee \dots)$, where w_i is true for document D_j when $t_i \in D_j$. So, the main operation of Boolean retrieval is to test if a term is in a document.
- Advantages
 - Results are predictable, relatively easy to explain to users
 - Many different features can be incorporated (e.g., document date or type)
 - Efficient processing since many documents can be eliminated from search
- Disadvantages
 - Effectiveness depends entirely on user queries (simple queries will not work well).
 - Complex queries are difficult that requires considerable experience.

2. Vector Space Model

- **Vector Space Models** are also classified as classical IR models.
- Documents and query are represented by a vector of term weights

$$Q = (q_1, q_2, \dots, q_t)$$

$$D_i = (d_{i1}, d_{i2}, \dots, d_{it})$$

- A Collection of Documents can be represented by a matrix of **term weights** (each row represents a document)

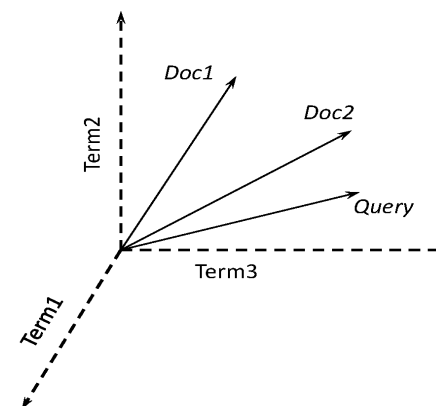
	<i>Term</i> ₁	<i>Term</i> ₂	...	<i>Term</i> _{<i>t</i>}
<i>Doc</i> ₁	<i>d</i> ₁₁	<i>d</i> ₁₂	...	<i>d</i> _{1<i>t</i>}
<i>Doc</i> ₂	<i>d</i> ₂₁	<i>d</i> ₂₂	...	<i>d</i> _{2<i>t</i>}
⋮	⋮			
<i>Doc</i> _{<i>n</i>}	<i>d</i> _{<i>n</i>1}	<i>d</i> _{<i>n</i>2}	...	<i>d</i> _{<i>n</i><i>t</i>}

- **Examples on the right-hand side:**
 - An **inverted index**, where each column represents a document.
 - A **3-dimensional visualization** illustrating only three term dimensions; note that such visualizations can be misleading when representing **high-dimensional spaces**.

CRICOS No.00213J

- 6
- D*₁ Tropical Freshwater Aquarium Fish.
*D*₂ Tropical Fish, Aquarium Care, Tank Setup.
*D*₃ Keeping Tropical Fish and Goldfish in Aquariums, and Fish Bowls.
*D*₄ The Tropical Tank Homepage - Tropical Fish and Aquariums.

Terms	Documents			
	<i>D</i> ₁	<i>D</i> ₂	<i>D</i> ₃	<i>D</i> ₄
aquarium	1	1	1	1
bowl	0	0	1	0
care	0	1	0	0
fish	1	1	2	1
freshwater	1	0	0	0
goldfish	0	0	1	0
homepage	0	0	0	1
keep	0	0	1	0
setup	0	1	0	0
tank	0	1	0	1
tropical	1	1	1	2



Vector Space Model - similarity measure

- Documents ranked by distance between points representing query and documents
 - Similarity* measure more common than a distance or *dissimilarity* measure
 - e.g., Cosine correlation equation:

$$\text{Cosine}(D_i, Q) = \frac{\sum_{j=1}^t d_{ij} \cdot q_j}{\sqrt{\sum_{j=1}^t d_{ij}^2 \cdot \sum_{j=1}^t q_j^2}}$$

$$Q = (q_1, q_2, \dots, q_t)$$

$$D_i = (d_{i1}, d_{i2}, \dots, d_{it})$$

- Examples of cosine similarity:
considering two documents D_1, D_2 and a query Q
 - $D_1 = (0.5, 0.8, 0.3), D_2 = (0.9, 0.4, 0.2),$
 - $Q = (1.5, 1.0, 0)$

$$\begin{aligned} \text{Cosine}(D_1, Q) &= \frac{(0.5 \times 1.5) + (0.8 \times 1.0)}{\sqrt{(0.5^2 + 0.8^2 + 0.3^2)(1.5^2 + 1.0^2)}} \\ &= \frac{1.55}{\sqrt{(0.98 \times 3.25)}} = 0.87 \end{aligned}$$

$$\begin{aligned} \text{Cosine}(D_2, Q) &= \frac{(0.9 \times 1.5) + (0.4 \times 1.0)}{\sqrt{(0.9^2 + 0.4^2 + 0.2^2)(1.5^2 + 1.0^2)}} \\ &= \frac{1.75}{\sqrt{(1.01 \times 3.25)}} = 0.97 \end{aligned}$$

Document Representation - *Term Weights*

- *tf*idf* weight

- Term frequency (*tf*) weight measures importance of term k in document (D_i):
$$tf_{ik} = \frac{f_{ik}}{\sum_{j=1}^t f_{ij}}$$

- Inverse document frequency (*idf*) measures importance in collection:
$$idf_k = \log \frac{N}{n_k}$$

- Where f_{ik} (or f_{ij}) is the number of occurrences of term k (or term j) in D_i , n_k is the number of documents in which term k occurs, and N is the total number of documents.

- A heuristic modification of *tf*idf* weight

$$d_{ik} = \frac{(\log(f_{ik}) + 1) \cdot \log(N/n_k)}{\sqrt{\sum_{x=1}^t [(\log(f_{ix}) + 1) \cdot \log(N/n_x)]^2}}$$

- Interpreting the mathematical expression d_{ik}

- The notation d_{ik} denotes the value of a Python function

def $d(i, k)$:

where i and k are **formal parameters** corresponding to the function inputs, with i indexing the document and k indexing the term dimension.

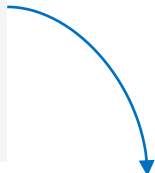
Example of Document representation in Python

- The document collection is represented as a Python dictionary, where each document (e.g., D1 is represented as a set of *term:frequency* pairs (e.g., {'Tropical':1, 'Freshwater':1, 'Aquarium':1, 'Fish':1})

```
D = {'D1':{'Tropical':1, 'Freshwater':1, 'Aquarium':1, 'Fish':1}, 'D2':{'Tropical':1, 'Fish':1, 'Aquarium':1, 'Care':1, 'Tank':1, 'Setup':1}, 'D3':{'Keeping':1, 'Tropical':1, 'Fish':2, 'and':2, 'Goldfish':1, 'in':1, 'Aquariums':1, 'Bowls':1}, 'D4':{'The':1, 'Tropical':2, 'Tank':1, 'Homepage':1, 'Fish':1, 'and':1, 'Aquariums':1}}
```

- Print terms in each document:

```
for id, doc in D.items():
    document = id+": "
    for term in doc.keys():
        document = document + term + ", "
    document = document[:-2]
    print(document)
```



```
D1: Tropical, Freshwater, Aquarium, Fish
D2: Tropical, Fish, Aquarium, Care, Tank, Setup
D3: Keeping, Tropical, Fish, and, Goldfish, in, Aquariums, Bowls
D4: The, Tropical, Tank, Homepage, Fish, and, Aquariums
```

Example cont.

Define a Python function to calcult *df* for each term

- The function starts to initailize an empty dictionary `df_`.
- It then finds each term using for loops, and then accumulate the *df* by using *try: except* structure.

```
def c_df(docs):  
    df_ = {}  
    for id, doc in docs.items():  
        for term in doc.keys():  
            try:  
                df_[term] += 1  
            except KeyError:  
                df_[term] = 1  
    return df_
```

Display document frequency of all terms:

```
{'Aquarium': 2, 'Aquariums': 2, 'Bowls': 1, 'Care': 1, 'Fish': 4, 'Freshwater': 1, 'Goldfish': 1, 'Homepage': 1, 'Keeping': 1, 'Setup': 1,  
'Tank': 2, 'The': 1, 'Tropical': 4, 'and': 2, 'in': 1}
```

Vector Space Model cont.

- The main operation is to calculate the similarity between the query and a document.
- Advantages
 - Simple computational framework for ranking.
 - Any similarity measure or term weighting scheme could be used.
- Disadvantages
 - Assumption of term independence.
 - No *predictions* about techniques for effective ranking. There is an **implicit assumption that relevance** is related to the similarity of query and document vectors.

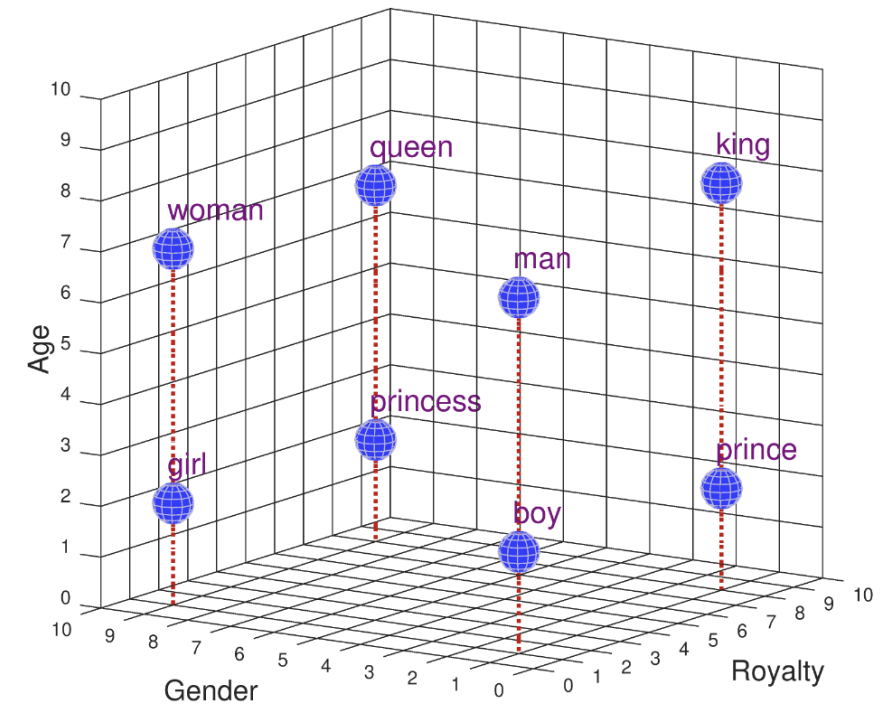
3. Word Embedding

- Word embedding technique is an unsupervised method for NLP and text analysis tasks, which is used for the representation of words in text.
- It describes term dependencies from a contextual perspective.
- It encodes the meaning of a word in the form of a real-valued vector, such that the words that are closer in the vector space are expected to be similar in meaning.
- For example, Word2vec embeddings in Python package *gensim*.
 - The word2vec algorithm uses a neural network model to learn word associations from a large corpus of text.
 - The word2vec algorithms include *skip-gram* and *CBOW* models, using either hierarchical softmax or negative sampling.
 - E.g., the vector of “machines” =
[-0.0366, -0.0750, 0.1216, -0.0339, 0.0837, 0.0024, 0.0983, 0.0981, -0.1342, 0.0736]

Semantic Feature Space

- Consider a 3D semantic feature space: Gender, Age and Royalty. We want to plot words (e.g., "king", "queen", "prince", "princess", "man", "woman", "boy", and "girl") in the 3D space.
 - Each word has three coordinate values: age, gender, and royalty. We call these lists of numbers *vectors*.
 - We can use a simpler method, *dot product*, to measure the similarity between two words $[x_1, y_1, z_1]$ and $[x_2, y_2, z_2]$:

$$[x_1, y_1, z_1] \cdot [x_2, y_2, z_2] = x_1 * x_2 + y_1 * y_2 + z_1 * z_2$$
- However, it is difficult to find a suitable semantic feature space that can contain many coordinate values (e.g., 300 dimensions), and it is also impossible to manually provide a vector for each word.



Word Coordinates

Gender Age Royalty

man	[	1,	7,	1	]
woman	[	9,	7,	1	]
boy	[	1,	2,	1	]
girl	[	9,	2,	1	]
king	[	1,	8,	8	]
queen	[	9,	7,	8	]
prince	[	1,	2,	8	]
princess	[	9,	2,	8	]

How to encode words automatically?

- There are different algorithms for word embeddings.
- Here we describe one of the first ones, called Skip-gram with Negative Sampling.

thou	shalt, not
shalt	thou, not, make
not	thou, shalt, make, a
make	shalt, not, a, machine
a	not, make, machine, in, image, of, human, mind
machine	make, a, in, the
in	a, machine, the, image
the	machine, in, image, of
image	in, the, of, a
of	the, image, a, human
human	of, a, mind
mind	a, human

1. Assemble a text corpus; and choose a vocabulary size M ;
2. Choose a context window size C (e.g., $C=2$, we will look at $2C+1 = 5$ word sliding window, 2 words to the left and 2 words to the right);
3. Assemble a co-occurrence dictionary (e.g., "*Thou shalt not make a machine in the image of a human mind*", see left table);
4. Choose an embedding size N (e.g., $N=300$) so that each word will be represented by a list of N numbers;
5. Make two tables each containing M rows and N columns: E and U ; and initialize both tables with small random numbers;
6. Slide a window of size $2C+1$ over the entire training corpus, shifting it by one word each time;

For each target word t (the middle slot in the window), look up its vector e in the embedding table E ;

For each context word u in the window (positive words),

look up its vector u in the context table U ; and compute $e \cdot u$ and run the result using "*sigmoid*" function (in range 0 to 1). If the output < 1 , make a tiny adjustment to e and u ;

For each negative words of t and apply the same procedure as the positive words (except we want the dot product to be 0 rather than 1);

7. Return E .

Bidirectional Encoder Representations from Transformers¹⁵ (BERT)

- BERT, a kind of LLM [5], is an efficient technique for word embedding, which is a pre-trained language model. It can also be used for supervised learning (e.g., classification) by fine-tuning.
- Loading BERT Pre-trained Model

```
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased')
```
- Tokenized Text (**An example** of how word “Machinelearning” is split into subwords):

```
['machine', '##lea', '##rn', '##ing', 'for', 'natural', 'language', 'processing']
```
- Encoded Text:

```
tensor([[ 101, 3698, 19738, 6826, 2075, 2005, 3019, 2653, 6364, 102]])
```
- Sentence Embedding:

```
tensor([[ -1.8279e-01, -2.9399e-01, -1.9029e-01, -1.3857e-01, 2.3039e-01, -6.5801e-02, -3.3547e-02, 5.0406e-01, -4.7368e-01, -1.6688e-01, -2.3568e-02, -9.0659e-02, -3.5038e-01, -6.0291e-02, -5.2360e-01, ..., -5.6265e-01, -2.1950e-01]])
```
- Shape of Sentence Embedding:

```
torch.Size([1, 768])
```

CRICOS No.00213J

```
import random
import torch from transformers
import BertTokenizer, BertModel from sklearn.metrics.pairwise

#Input text
text = "Machinelearning for Natural Language Processing"
encoding = tokenizer.batch_encode_plus([text], # List of input texts
padding=True, # Pad to the maximum sequence length
truncation=True, # Truncate to the maximum sequence length if necessary
return_tensors='pt', # Return PyTorch tensors
add_special_tokens=True # Add special tokens CLS and SEP
)
# Generate embeddings using BERT model
with torch.no_grad():
    outputs = model(input_ids, attention_mask=attention_mask)
    word_embeddings = outputs.last_hidden_state # it contains the embeddings

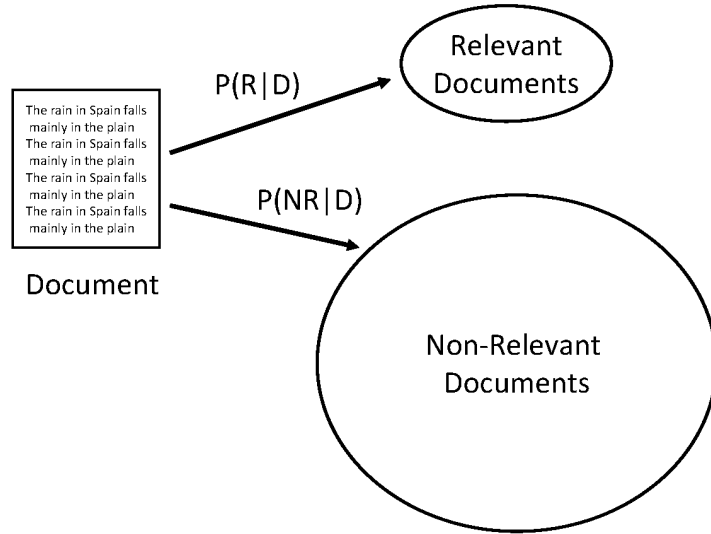
# Compute the average of word embeddings to get the sentence embedding
sentence_embedding = word_embeddings.mean(dim=1)
```

- **Word2Vec** and **BERT** are both representation learning models for natural language, but they differ fundamentally in their modeling assumptions, representational power, and downstream applicability.
- **Word2Vec**
 - Word2Vec is a shallow neural embedding model that learns **static word representations**. Each word in the vocabulary is mapped to a single fixed vector, independent of context. The model is trained using either the **Continuous Bag-of-Words (CBOW)** or **Skip-gram** objective, which captures local co-occurrence statistics.
 - **Key characteristics:**
 - Produces one embedding per word type
 - Ignores word order and long-range dependencies
 - Efficient to train and deploy
 - But limited ability to model polysemy and contextual meaning
- **BERT**
 - BERT is a deep **Transformer-based** language model that learns **contextualized token representations**. Each token embedding is dynamically generated based on its surrounding context using multi-head self-attention.
 - BERT is pretrained using **Masked Language Modeling** to capture bidirectional context; **Next Sentence Prediction** (in the original formulation) to model sentence-level coherence.
 - **Key characteristics:**
 - Produces different embeddings for the same word in different contexts
 - Captures long-range syntactic and semantic dependencies
 - Supports end-to-end fine-tuning for downstream tasks
 - But computationally expensive compared to static embedding models

4. Generative Models: Bayes classifier

- It is hard to prove that an IR model will achieve better effectiveness than any other model since we are trying to formalize a complex human activity.
- Now we discuss a theoretical idea for this hard problem:
 - **Generative** (e.g., Bayes classifier or probabilistic model) approach (the opposite is **discriminative** (e.g., logistic regression) approach).
- In statistics, a generative model is a joint probability distribution $P(X, Y)$ on a given observable variable X and target variable Y .
- It tries to generate random instances (outcomes) of an observation x , or a kind of probability function defined for sets of outcomes.
- **Probability Ranking Principle** (early theoretical statement about effectiveness, Robertson (1977)).
“Originally described as follow”:
 - “If a reference retrieval system’s response to each request is a **ranking of the documents** in the collection in order of decreasing probability of relevance to the user who submitted the request,
 - where **the probabilities** are estimated as accurately as possible on the basis of whatever data have been made available to the system for this purpose,
 - the overall effectiveness of the system to its user will be the best that is obtainable on the basis of those data.”

IR as Classification - Bayes Classifier



- $P(R|D)$ is a **conditional probability** representing the probability of relevance for the given document D , and
- $P(NR|D)$ is the conditional probability of non-relevance.

- Bayes Decision Rule
 - A document D is relevant if $P(R|D) > P(NR|D)$

- Estimating probabilities
 - Use **Bayes Rule**

$$P(R|D) = \frac{P(D|R)P(R)}{P(D)}$$

- $P(R)$ is the priori probability of relevance
- $P(D)$ is the probability of D 's occurrence
- Classify document D as relevant if $P(R|D) > P(NR|D)$, where

$$P(NR|D) = \frac{P(D|NR)P(NR)}{P(D)}$$

That is

$$\frac{P(D|R)}{P(D|NR)} > \frac{P(NR)}{P(R)}$$

- The left-hand side (lhs) is *likelihood ratio*.

Examples of Relevant and Non-relevant Documents

Document ID	Terms: d_{ij}	Relevance to Q
D ₁	GERMAN, VW	0 no
D ₂	US, US, ECONOM, SPY	1 yes
D ₃	US, BILL, ECONOM, ESPIONAG	1 yes
D ₄	US, ECONOM, ESPIONAG, BILL	1 yes
D ₅	GERMAN, MAN, VW, ESPIONAG	0 no
D ₆	GERMAN, GERMAN, MAN, VW, SPY	0 no
D ₇	US, MAN, VW	0 no

- Let $Q = \{\text{US, ECONOM, ESPIONAG}\}$ be a query
- $\mathcal{C} = \{D_1, D_2, D_3, D_4, D_5, D_6, D_7\}$ be a collection of documents, where
 - $D_1 = \{\text{GERMAN, VW}\}$
 - $D_2 = \{\text{US, US, ECONOM, SPY}\}$
 - $D_3 = \{\text{US, BILL, ECONOM, ESPIONAG}\}$
 - $D_4 = \{\text{US, ECONOM, ESPIONAG, BILL}\}$
 - $D_5 = \{\text{GERMAN, MAN, VW, ESPIONAG}\}$
 - $D_6 = \{\text{GERMAN, GERMAN, MAN, VW, SPY}\}$
 - $D_7 = \{\text{US, MAN, VW}\}$
- $T = \{\text{US, BILL, ECONOM, ESPIONAG, GERMAN, MAN, VW, SPY}\} = \{d_1, d_2, d_3, d_4, d_5, d_6, d_7, d_8\}$
- $P(R|D_5) = ?, P(R|D) = ?$ Where $D = \{\text{US, VW, ESPIONAG}\} = \{d_1, d_7, d_4\}$
- By using the Bayes rule, we have $P(R|D) = \frac{P(R)}{P(D)} P(D|R)$
- How to calculate $P(D|R)$?

Estimating $P(D|R)$

- **Assume term independence.** Under this assumption, we can derive an equation for the probability that a document D occurs in relevant documents:

$$P(D|R) = \prod_{i=1}^t P(d_i|R)$$

- **Example cont.**

- From the previous example, there are 8 terms. We define the vocabulary (term set) as

$$\begin{aligned} T &= \{US, BILL, ECONOM, ESPIONAG, GERMAN, MAN, VW, SPY\} \\ &= \{d_1, d_2, d_3, d_4, d_5, d_6, d_7, d_8\} \end{aligned}$$

- Let D be a document containing the terms, i.e.,

$$D = \{US, VW, ESPIONAG\}.$$

- A simple way to represent D is as a **binary feature vector**:

$D = (1, 0, 0, 1, 0, 0, 1, 0)$, where a value of 1 indicates that the corresponding term occurs in D , and 0 indicates its absence. For example, the first 1 means that D contains term $d_1 = US$.

- This representation illustrates the **Binary Independence Model**, in which each document is represented as a vector of binary features indicating term occurrence (or non-occurrence), and terms are assumed to be statistically independent given relevance.
- The remaining question is how to estimate $P(d_i|R)$, the probability that term d_i occurs in a relevant document, to compute the overall relevance probability of D .

4. Binary Independence Model

- Let $p_i = P(d_i | R)$ be the probability that term d_i occurs (i.e., has value 1) in relevant documents, and $s_i = P(d_i | NR)$ be the probability of d_i 's occurrence in non-relevant documents.

$$\begin{aligned}
 \frac{P(D|R)}{P(D|NR)} &= \prod_{i:d_i=1} \frac{p_i}{s_i} \cdot \prod_{i:d_i=0} \frac{1-p_i}{1-s_i} \\
 &= \prod_{i:d_i=1} \frac{p_i}{s_i} \cdot \left(\prod_{i:d_i=1} \frac{1-s_i}{1-p_i} \cdot \prod_{i:d_i=1} \frac{1-p_i}{1-s_i} \right) \cdot \prod_{i:d_i=0} \frac{1-p_i}{1-s_i} \\
 &= \prod_{i:d_i=1} \frac{p_i(1-s_i)}{s_i(1-p_i)} \cdot \prod_i \frac{1-p_i}{1-s_i}
 \end{aligned}$$

Where $i:d_i=1$ means terms that have the value 1 (presence) in document D ; and $i:d_i=0$ means terms that have the value 0 (absence) in document D .

- Example** (continued for document $D = \{\text{US, VW, ESPIONAG}\}$ with its binary vector (1, 0, 0, 1, 0, 0, 1, 0))
 - Based on the above independent assumption, we have
 - $P(D|R) = p_1 * (1-p_2) * (1-p_3) * p_4 * (1-p_5) * (1-p_6) * p_7 * (1-p_8) = (p_1 * p_4 * p_7) * [(1-p_2) * (1-p_3) * (1-p_5) * (1-p_6) * (1-p_8)]$
 - $P(D|NR) = s_1 * (1-s_2) * (1-s_3) * s_4 * (1-s_5) * (1-s_6) * s_7 * (1-s_8) = (s_1 * s_4 * s_7) * [(1-s_2) * (1-s_3) * (1-s_5) * (1-s_6) * (1-s_8)]$

Binary Independence Model cont.

- A scoring function (if we ignore the common 2^{nd} product for all documents), where we log the *likelihood ratios*:

$$\sum_{i:d_i=1} \log \frac{p_i(1-s_i)}{s_i(1-p_i)}$$

- Query provides information about relevant documents
- If we assume p_i constant, s_i approximated by entire collection, get *idf*-like weight for each term i :

$$\log \frac{0.5(1-\frac{n_i}{N})}{\frac{n_i}{N}(1-0.5)} = \log \frac{N-n_i}{n_i}$$

- Where n_i is the number of documents that contain term i and N is the total number of documents in the collection.
- But how to calculate p_i and s_i if we consider terms different distributions?

Contingency Table

- **A contingency table** is used to understand the terms distribution (presence and absence) in relevant and non-relevant documents.

	Relevant	Non-relevant	Total
$d_i = 1$	r_i	$n_i - r_i$	n_i
$d_i = 0$	$R - r_i$	$N - n_i - R + r_i$	$N - n_i$
Total	R	$N - R$	N

r_i is the number of relevant documents containing term i
 n_i is the number of documents containing term i
 N is the total number of documents in the collection
 R is the number of relevant documents for this query
 $d_i = 1$ if term i is present in the document, and 0 otherwise

$$p_i = (r_i + 0.5)/(R + 1) \quad s_i = (n_i - r_i + 0.5)/(N - R + 1)$$

- Putting these estimates into the scoring function:

$$\sum_{i: d_i = q_i = 1} \log \frac{(r_i + 0.5)/(R - r_i + 0.5)}{(n_i - r_i + 0.5)/(N - n_i - R + r_i + 0.5)}$$

Example of Contingency Table

- For the first term $i = 1$, 'US', let $d_1 = 1$ (terms distribution in relevant documents)
 - $r_1 = 3$ is the number of relevant documents containing 'US' (term 1)
 - $n_1 = 4$ is the number of documents containing 'US' (term 1)
 - $(n_1 - r_1) = 4 - 3 = 1$ is the number of non-relevant documents containing 'US' (term 1)
- For the first term $i = 1$, 'US', let $d_1 = 0$ (terms distribution in non-relevant documents)
 - $R - r_1 = 3 - 3 = 0$ is the number of relevant documents that don't contain 'US' (term 1)
 - $N - n_1 = 7 - 4 = 3$ is the number of documents that don't contain 'US' (term 1)
 - $(N - R) - (n_1 - r_1) = N - n_1 - R + r_1 = 7 - 4 - 3 + 3 = 3$ is the number of non-relevant documents that don't contain 'US' (term 1)

	Relevant	Non-relevant	Total
$d_1 = 1$	$r_1 = 3$	$n_1 - r_1 = 1$	$n_1 = 4$
$d_1 = 0$	$R - r_1 = 0$	$(N - R) - (n_1 - r_1) = N - n_1 - R + r_1 = 3$	$N - n_1 = 3$
Total	$R = 3$	$N - R = 4$	$N = 7$

5. The BM25 Ranking Algorithm

- BM25 extends the scoring function for the binary independence model to include document and query term weights.
- The extension is based on probabilistic arguments and experimental validation, but it is not a formal model.

$$\sum_{i \in Q} \log \frac{(r_i + 0.5) / (R - r_i + 0.5)}{(n_i - r_i + 0.5) / (N - n_i - R + r_i + 0.5)} \cdot \frac{(k_1 + 1) f_i}{K + f_i} \cdot \frac{(k_2 + 1) q f_i}{k_2 + q f_i}$$

- f_i is the frequency weight (count) of term i in the document;
- qf_i is the frequency weight (count) of term i in the query;
- k_1 , k_2 and K are parameters whose values are set empirically
- $K = k_1 \left((1 - b) + b \cdot \frac{dl}{avdl} \right)$, dl is doc length and $avdl$ is the average length of a document in the collection.
- Typical value for k_1 is 1.2, k_2 varies from 0 to 1000, $b = 0.75$

BM25 Example

- Query with two terms, “president lincoln”, ($qf = 1$)
- No relevance information (r and R are zero)
- $N = 500,000$ documents
- “*president*” occurs in 40,000 documents ($n_1 = 40,000$)
- “*lincoln*” occurs in 300 documents ($n_2 = 300$)
- “*president*” occurs 15 times in doc ($f_1 = 15$)
- “*lincoln*” occurs 25 times ($f_2 = 25$)
- document length is 90% of the average length ($dl/avdl = .9$)
- $k_1 = 1.2$, $b = 0.75$, and $k_2 = 100$
- $K = 1.2 \cdot (0.25 + 0.75 \cdot 0.9) = 1.11$

BM25 Example

$$\begin{aligned}
 BM25(Q, D) &= \log \frac{(0 + 0.5)/(0 - 0 + 0.5)}{(40000 - 0 + 0.5)/(500000 - 40000 - 0 + 0 + 0.5)} \\
 &\quad \times \frac{(1.2 + 1)15}{1.11 + 15} \times \frac{(100 + 1)1}{100 + 1} \\
 &\quad + \log \frac{(0 + 0.5)/(0 - 0 + 0.5)}{(300 - 0 + 0.5)/(500000 - 300 - 0 + 0 + 0.5)} \\
 &\quad \times \frac{(1.2 + 1)25}{1.11 + 25} \times \frac{(100 + 1)1}{100 + 1} \\
 &= \log 460000.5/40000.5 \cdot 33/16.11 \cdot 101/101 \\
 &\quad + \log 499700.5/300.5 \cdot 55/26.11 \cdot 101/101 \\
 &= 2.44 \cdot 2.05 \cdot 1 + 7.42 \cdot 2.11 \cdot 1 \\
 &= 5.00 + 15.66 = 20.66
 \end{aligned}$$

BM25 Example

- Effect of term frequencies

Frequency of “president”	Frequency of “lincoln”	BM25 score
15	25	20.66
15	1	12.74
15	0	5.00
1	25	18.2
0	25	15.66

- To summarize, BM25 is an effective ranking algorithm derived from a model of information retrieval viewed as classification.
- This model focuses on topical relevance and makes an explicit assumption that relevance is binary.

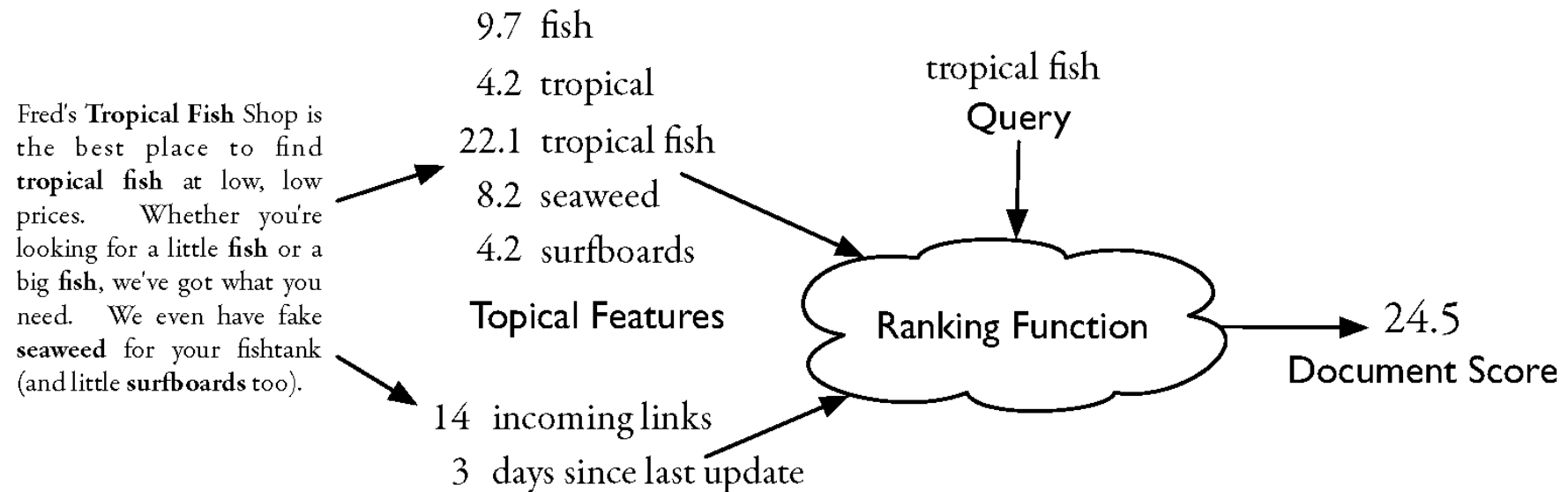
6. Abstract Model of Ranking for IR and ML

- In summary

- Documents are written in natural human languages, which are inherently difficult for computers to analyze directly.
- A common approach to represent documents is to select a set of **document features** (or **index terms**), where each feature corresponds to an attribute of the document that can be expressed numerically.
- Accurately modeling the probabilistic relationships between terms is challenging; therefore, most traditional **information retrieval (IR)** and **machine learning (ML)** models assume **term independence**.
- A more advanced alternative is to model term relationships using **word embeddings**, which are learned by pre-training on very large text corpora. In this representation, terms are mapped to dense numerical vectors, enabling the computation of semantic similarity between terms.

Examples of Document Features

- Document features categories: Topical features (e.g., important keywords) and Quality features (e.g., meta information, such as days since last update of the document).
 - Each of these feature values is generated using a feature function (e.g., tf, df, tf*idf, or BM25 scoring function).
- The following figure shows the components of the abstract model of ranking: documents, features, queries, the ranking (retrieval) function, and document scores.
- The ranking function takes data from document features combined with the query and produces a score.



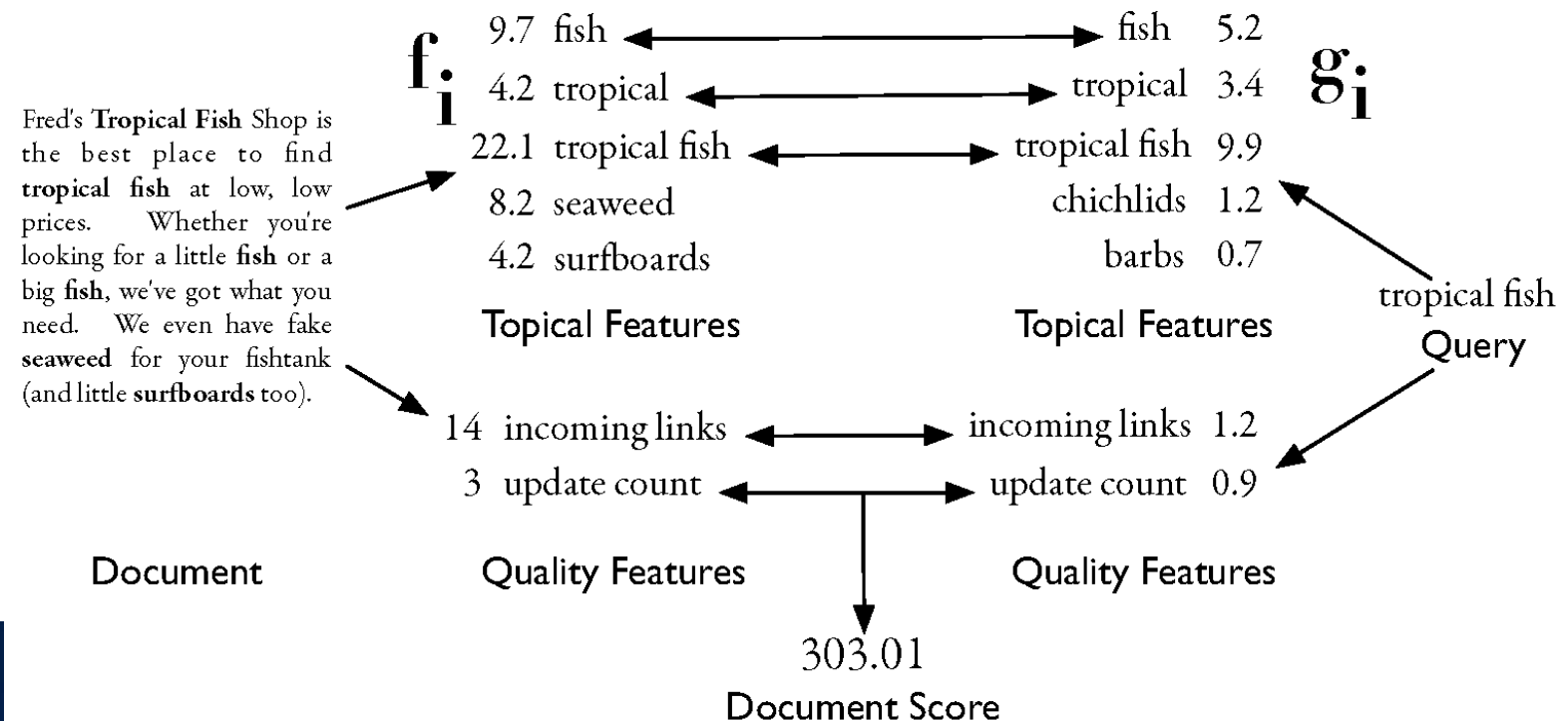
A Ranking Function for IR

- We may assume that the ranking function R takes the following form:

$$R(Q, D) = \sum_i g_i(Q) f_i(D)$$

f_i is a document feature function
 g_i is a query feature function

- Each pair of feature functions is multiplied together, and the results from all pairs are added to create a final document score.



References

- [1] Chapter 7 (sections 7.1, 7.2 and 7.3) in book - W. Bruce Croft, *Search Engines - Information retrieval in Practice*; Pearson, 2010.
- [2] Chapter 1, "Probability", 1993, ISBN : 978-1-4612-6937-3,
https://qut.primo.exlibrisgroup.com/permalink/61QUT_INST/1g7tbfa/alma991010463496504001
- [3] Albishre K., Li Y., Xu Y. (2018) *Query-Based Automatic Training Set Selection for Microblog Retrieval*. In: Phung D., Tseng V., Webb G., Ho B., Ganji M., Rashidi L. (eds) *Advances in Knowledge Discovery and Data Mining*. PAKDD 2018. Lecture Notes in Computer Science, vol 10938. DOI https://doi.org/10.1007/978-3-319-93037-4_26
- [4] Albishre, K., Li, Y. *et al.* Query-based unsupervised learning for improving social media search. *World Wide Web* **23**, 1791–1809 (2020). <https://doi.org/10.1007/s11280-019-00747-0>
- [5] Larisa Gorenstein, Eli Konen, Michael Green, Eyal Klang, *Bidirectional Encoder Representations from Transformers in Radiology: A Systematic Review of Natural Language Processing Applications*, Journal of the American College of Radiology, Volume 21, Issue 6, 2024, Pages 914-941, <https://doi.org/10.1016/j.jacr.2024.01.012>